

Avviso per tutti i docenti e i responsabili dei corsi

A seguito di un importante aggiornamento della piattaforma, attualmente in corso, vi chiediamo di **non duplicare i quiz contenenti domande random** fino a nuova comunicazione.

Questa misura è necessaria per consentirci di completare l'aggiornamento e risolvere le problematiche riscontrate nel minor tempo possibile.

Vi ringraziamo per la collaborazione.



Ingegneria del Software

[Torna al corso](#)

Stato	Completato
Iniziato	giovedì, 25 giugno 2026, 19:19
Terminato	giovedì, 25 giugno 2026, 19:31
Tempo impiegato	12 min. 33 secondi
Punteggio	50,00/50,00
Valutazione	100,00 su un massimo di 100,00

Domanda 1

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo di sviluppo agile consiste di varie iterazioni. Alla fine di ogni iterazione vengono prese in considerazione le "change requests" e, se ve ne sono, l'iterazione viene ripetuta. Sia p la probabilità che ci siano "change requests" all fine di una iterazione e sia A il costo di una iterazione. Il valore atteso del costo per l'iterazione è:

Scegli un'alternativa:

- ☐ 1. A
- ☐ 2. $p \cdot A$
- ☒ 3. $(1 + p) \cdot A$ ✓

Your answer is correct.

La risposta corretta è: $(1 + p) \cdot A$

Domanda 2

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il team di sviluppo di un'azienda consiste di un senior software engineer e due sviluppatori junior. Usando un approccio plan-driven (ad esempio, water-fall) la fase di design impegna solo il membro senior per tre mesi e la fase di sviluppo e testing solo i due membri junior per tre mesi. Si assuma che non ci siano "change requests" e che il membro senior costi A Eur/mese ed i membri junior B Eur/mese. Qual'è il costo dello sviluppo usando un approccio plan-driven come sopra ?

Scegli un'alternativa:

- ☐ 1. $3*A + 3*B$
- ☒ 2. $3*A + 6*B$ ✓
- ☐ 3. $A + 2*B$

Your answer is correct.

La risposta corretta è: $3*A + 6*B$ **Domanda 3**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il rischio R può essere calcolato come $R = P*C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso.

Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = \exp(-b*S)$, dove b è una opportuna costante note da dati storici aziendali.

- $b = 0.000075$
- $C = 731116.000000$
- $R = 541.000000$

Quale delle seguenti opzioni meglio approssima il costo S per lo sviluppo del software in questione.

Scegli un'alternativa:

- ☐ 1. Costo: 144265
- ☐ 2. Costo: 60305
- ☒ 3. Costo: 96358 ✓ Correct.

Your answer is correct.

La risposta corretta è: Costo: 96358

Domanda 4

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x_1 rappresenta lo stato precedente ad x e la variabile x_2 lo stato precedente ad x_1 .

Si consideri il seguente requisito:

Se lo stato vale 0 per due transizioni consecutive, allora alla transizione successiva lo stato avrà valore 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( (x2 == 0) && (x1 == 0) ? (x == 1) : 0 )`
- ☐ b. `assert( (x == 0) && (x1 == 0) ? (x2 == 1) : 1 )`
- ☒ c. `assert( (x2 == 0) && (x1 == 0) ? (x == 1) : 1 )` ✓

Your answer is correct.

La risposta corretta è:

`assert( (x2 == 0) && (x1 == 0) ? (x == 1) : 1 )`

Domanda 5

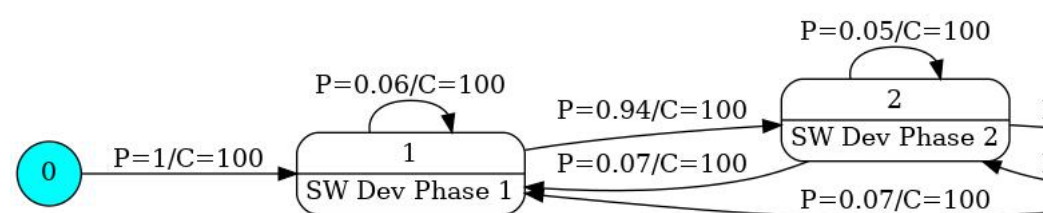
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

Scegli un'alternativa:

- ☒ 1. 0.0856 ✓ Correct.
- ☐ 2. 0.0437324
- ☐ 3. 0.032739

Your answer is correct.

La risposta corretta è: 0.0856

Domanda 6

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile **x** rappresenta lo stato presente del sistema, la variabile **x1** rappresenta lo stato precedente ad **x** e la variabile **x2** lo stato precedente ad **x1**.

Si consideri il seguente requisito:

Se lo stato vale 0 allora dopo al più 2 transizioni lo stato prende il valore 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☒ a. `assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 1 )` ✓
- ☐ b. `assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 0 )`
- ☐ c. `assert( (x == 0) ? (x1 == 1) || (x2 == 1) : 1 )`

Your answer is correct.

La risposta corretta è:

`assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 1 )`

Domanda 7

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo di sviluppo plan-driven consiste di 2 fasi **F1**, **F2**, ciascuna costo **A**. Alla fine di ogni fase vengono prese in considerazione le "change requests" e, se ve ne sono, lo sviluppo viene ripetuto a partire dalla prima iterazione. Quindi con nessuna change request si hanno le fasi: **F1**, **F2** e costo **2A**. Con una "change request" dopo la prima fase si ha: **F1**, **F1**, **F2** e costo **3A**. Con una change request dopo la fase 2 si ha: **F1**, **F2**, **F1**, **F2** e costo **4A**. Qual'è il costo nel caso in cui ci siano change requests sia dopo la fase 1 che dopo la fase 2.

Scegli un'alternativa:

- ☐ 1. **6*A**
- ☐ 2. **7*A**
- ☒ 3. **5*A** ✓

Your answer is correct.

Nel caso in cui ci siano change requests sia dopo la fase 1 che dopo la fase 2 si ha: **F1**, **F1**, **F2**, **F1**, **F2**. Il costo quindi è **5*A**.

La risposta corretta è: **5*A**

Domanda 8

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di consistenza" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che per ogni requisito esista un insieme di test che lo possa verificare.
- ☒ 2. Assicurarsi che non ci siano requisiti in conflitto con altri requisiti. ✓
- ☐ 3. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.

Your answer is correct.

La risposta corretta è: Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.

Domanda 9

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Focalizzandosi sui metodi agile di sviluppo del software, quale delle seguenti affermazioni è vera?

Scegli un'alternativa:

- ☐ 1. Per evitare di sprecare tempo durante la fase di sviluppo del software, il customer non è mai coinvolto nel processo di sviluppo del software.
- ☐ 2. Per evitare di sprecare tempo durante la fase di sviluppo del software, questa inizia solo quando i requisiti sono stati completamente definiti.
- ☒ 3. Le attività di definizione dei requisiti e di sviluppo sono interleaved. ✓

Your answer is correct.

La risposta corretta è: Le attività di definizione dei requisiti e di sviluppo sono interleaved.

Domanda 10

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri il seguente frammento di pseudo codice.

```
// Begin input/output connections for system components
```

```
Input 12 of SC2 = Output 12 of SC1
```

```
Input 21 of SC1 = Output 21 of SC2
```

```
Input 23 of SC3 = Output 23 of SC2
```

```
Input 24 of SC4 = Output 24 of SC2
```

```
Input 32 of SC2 = Output 32 of SC3
```

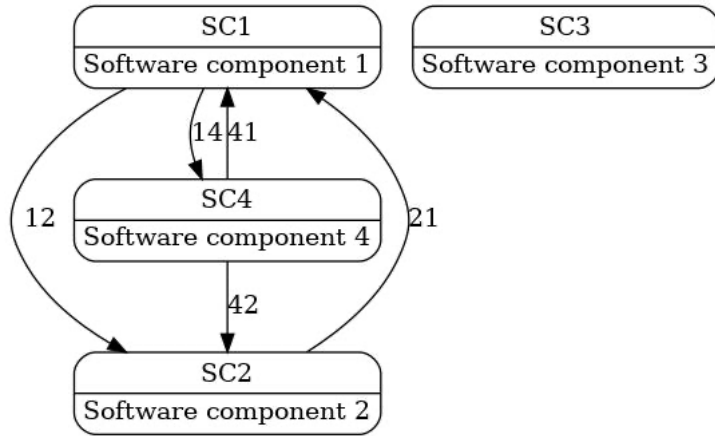
```
Input 43 of SC3 = Output 43 of SC4
```

```
// End input/output connections for system components
```

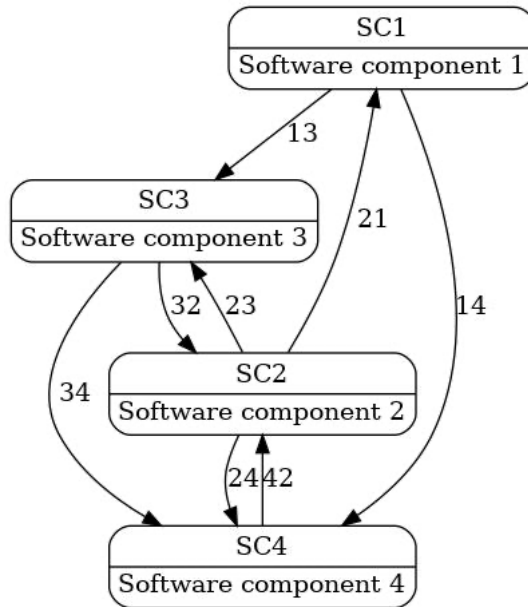
Quale delle seguenti architetture software meglio lo rappresenta ?

Scegli un'alternativa:

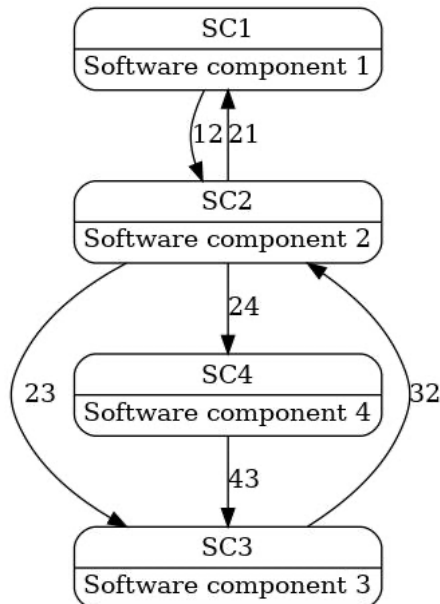
1.



2.

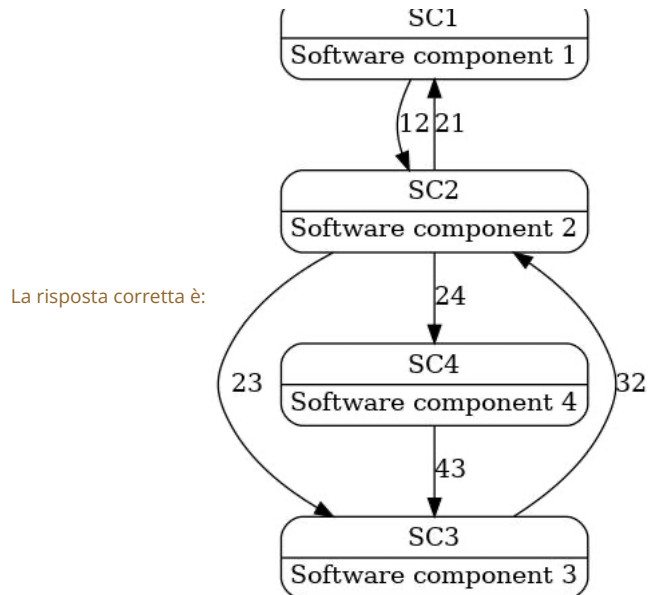


3.



Correct.

Your answer is correct.



Domanda 11

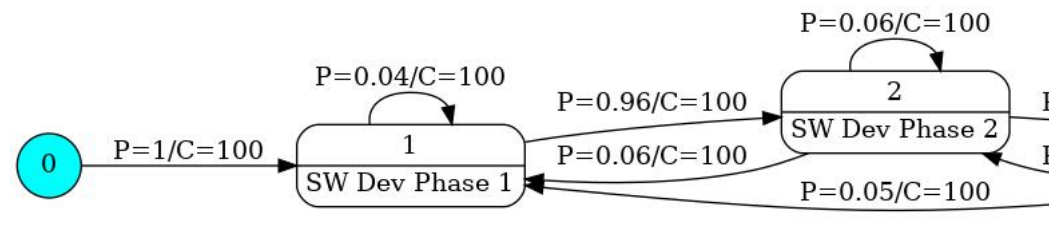
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Scegli un'alternativa:

- ☒ 1. 0.76032 ✔ Correct.
- ☐ 2. 0.499677
- ☐ 3. 0.232579

Your answer is correct.

La risposta corretta è: 0.76032

Domanda 12

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo di sviluppo agile consiste di 3 iterazioni identiche di costo A. Alla fine di ogni iterazione vengono prese in considerazione le "change requests" e, se ve ne sono, l'iterazione viene ripetuta. Sia p la probabilità che ci siano "change requests" all fine di una iterazione. Il valore atteso del costo del progetto è:

Scegli un'alternativa:

- ☐ 1. $3 \cdot p \cdot A$
- ☒ 2. $3 \cdot (1 + p) \cdot A$ ✓
- ☐ 3. $3 \cdot (A + p)$

Your answer is correct.

La risposta corretta è: $3 \cdot (1 + p) \cdot A$

Domanda 13

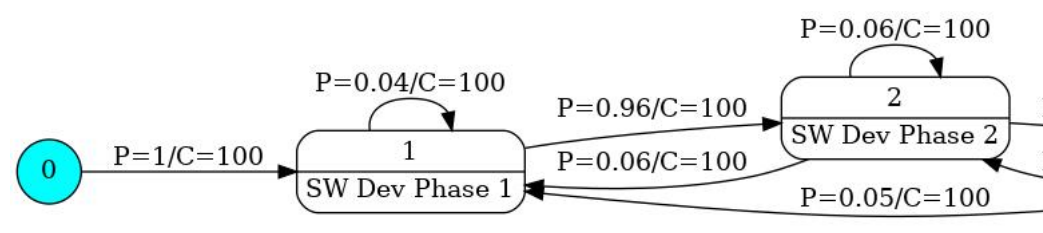
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.0223997
- ☒ 2. 0.0969 ✓ Correct.
- ☐ 3. 0.0516906

Your answer is correct.

La risposta corretta è: 0.0969

Domanda 14

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri un software sviluppato seguendo un approccio iterativo implementato con due fasi: F1 seguita da F2. Ciascuna fase ha costo 1000 Eur e deve essere ripetuta una seconda volta con probabilità 0.5. Qual'è il costo atteso dello sviluppo dell'intero software?

Scegli un'alternativa:

- ☒ 1. 3000 ✓
- ☐ 2. 2000
- ☐ 3. 5000

Your answer is correct.

La risposta corretta è: 3000

Domanda 15

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x_1 rappresenta lo stato precedente ad x e la variabile x_2 lo stato precedente ad x_1 .

Si consideri il seguente requisito:

La differenza tra il valore dello stato presente e di quello precedente è compresa tra -1 ed 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( ( x - x1) <= 1 )`
- ☒ b. `assert( (-1 <= (x - x1)) && ((x - x1) <= 1) )` ✓
- ☐ c. `assert( (-1 <= (x - x1)) && ((x1 - x) <= 1) )`

Your answer is correct.

La risposta corretta è:

`assert( (-1 <= (x - x1)) && ((x - x1) <= 1) )`**Domanda 16**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente funzione di transizione.

```

#include "main.h"

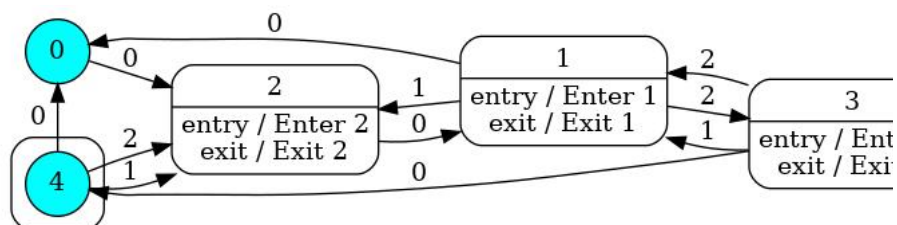
int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (1);
    case 1: return (2);
    case 2: return (1);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (4);
    case 1: return (2);
    case 2: return (1);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (0);
    case 1: return (0);
    case 2: return (3);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (2);
    case 1: return (3);
    case 2: return (2);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (4);
    case 1: return (2);
    case 2: return (0);
    default: return (0); }
  default: return (0); }
} // f()

```

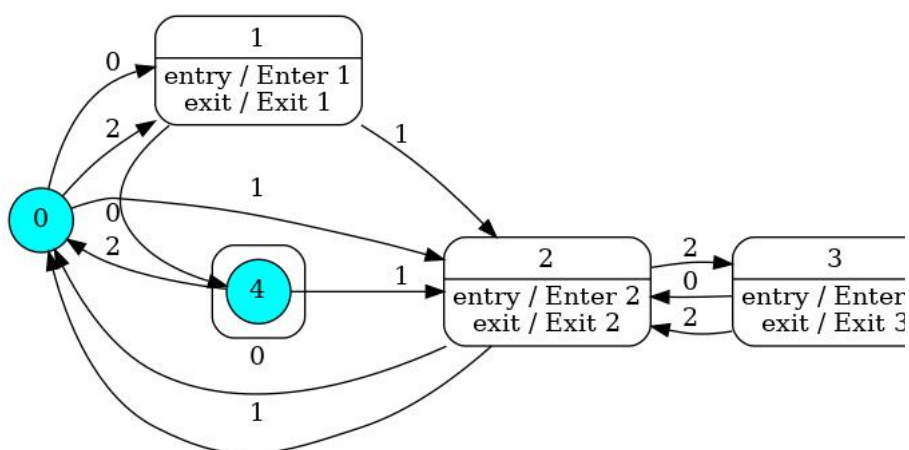
Quale dei seguenti UML state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?

Scegli un'alternativa:

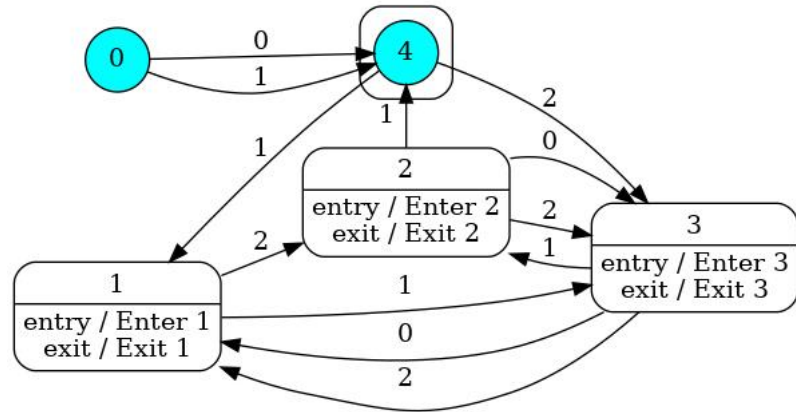
☐ 1.



☒ 2.

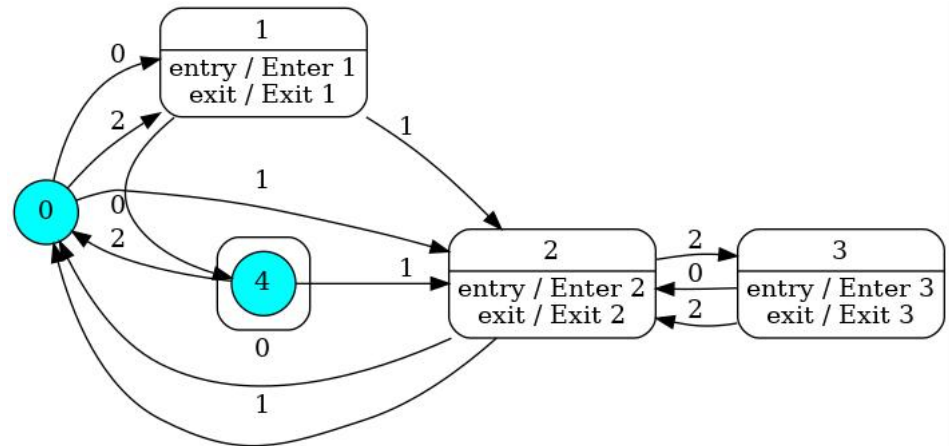


5.



Your answer is correct.

La risposta corretta è:



Domanda 17

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.

La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la path coverage con i tests cases dati.

```
int f(int x[3])
{
    if ((2)*x[0] + (0)*x[1] + (-2)*x[2] <= 1)
        if ((-2)*x[0] + (0)*x[1] + (-4)*x[2] <= -1)
            {return (0); }
        else
            {return (1); }
    else if ((0)*x[0] + (2)*x[1] + (-2)*x[2] <= 2)
        if ((-2)*x[0] + (0)*x[1] + (-2)*x[2] <= -3)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */
```

Test 1: x[0] = 5, x[1] = -1, x[2] = 2.

Test 2: x[0] = 1, x[1] = -3, x[2] = 5.

Test 3: x[0] = -1, x[1] = 1, x[2] = -2.

Test 4: x[0] = -4, x[1] = -3, x[2] = -3.

Scegli un'alternativa:

- ☐ 1. Coverage: 0.539
- ☐ 2. Coverage: 0.208
- ☒ 3. Coverage: 0.800  Correct.

Your answer is correct.

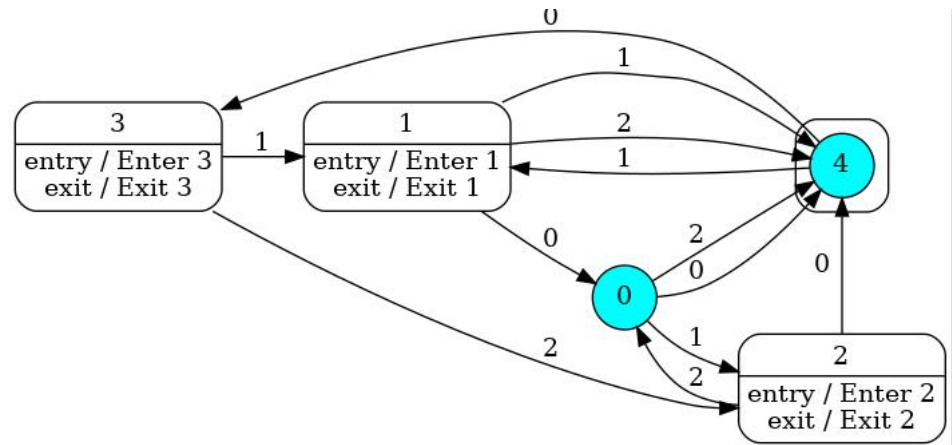
La risposta corretta è: Coverage: 0.800

Domanda 18

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri lo state diagram in figura.



Quale delle seguenti funzioni di transizione meglio lo rappresenta?

Scegli un'alternativa:

- ☒ 1. `#include "main.h"` ✔ Correct.

```
int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (4);
    case 1: return (2);
    case 2: return (4);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (0);
    case 1: return (4);
    case 2: return (4);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (4);
    case 1: return (2);
    case 2: return (0);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (3);
    case 1: return (1);
    case 2: return (2);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (3);
    case 1: return (1);
    case 2: return (4);
    default: return (0); }
  default: return (0);
} } // f()
```

☐ 2. `#include "main.h"`

```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (4);
        case 1: return (3);
        case 2: return (0);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (3);
        case 1: return (4);
        case 2: return (0);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (3);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (3);
        case 1: return (2);
        case 2: return (3);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (1);
        case 1: return (0);
        case 2: return (0);
        default: return (0); }
    default: return (0);
}
} // f()
```

☐ 3. `#include "main.h"`

```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (2);
        case 1: return (0);
        case 2: return (4);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (0);
        case 1: return (4);
        case 2: return (2);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (3);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (2);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (0);
        case 1: return (4);
        case 2: return (2);
        default: return (0); }
    default: return (0);
}
} // f()
```

Your answer is correct.

La risposta corretta è:

```
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (4);
        case 1: return (2);
        case 2: return (4);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (0);
        case 1: return (4);
        case 2: return (4);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (2);
        case 2: return (0);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (2);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (4);
        default: return (0); }
    default: return (0);
}
} // f()
```

Domanda 19

Si consideri la seguente funzione di transizione.

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

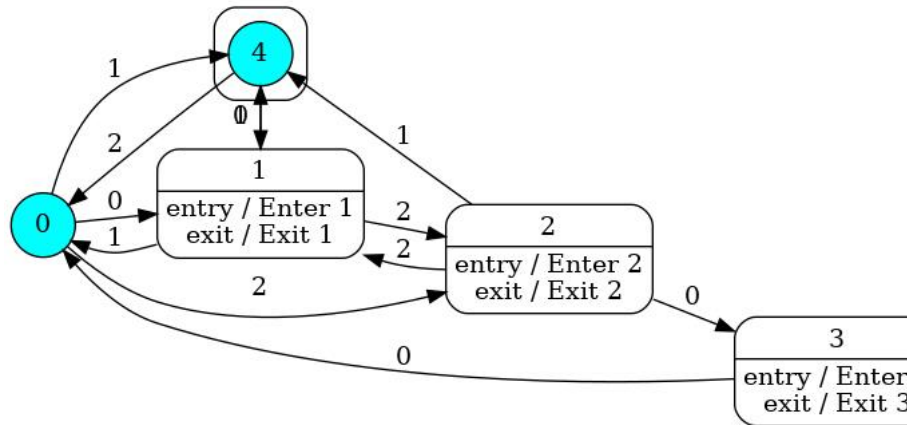
```
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (2);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (4);
        case 1: return (0);
        case 2: return (2);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (3);
        case 1: return (4);
        case 2: return (1);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (0);
        case 1: return (3);
        case 2: return (3);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    default: return (0);
}
} // f()
```

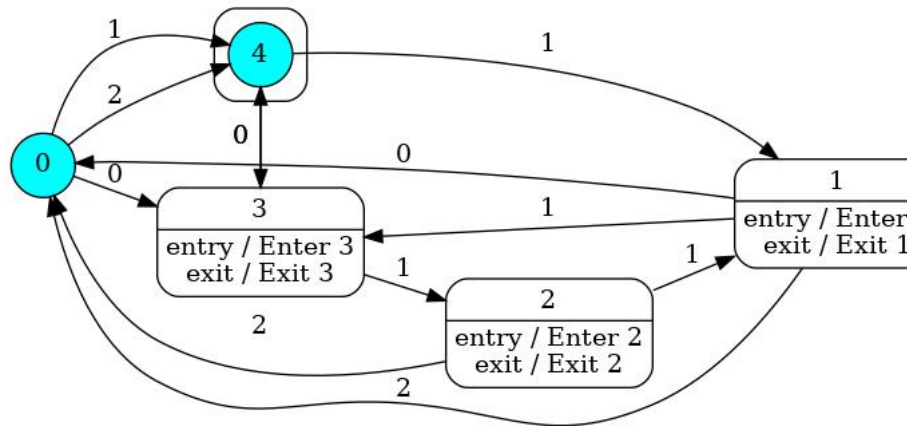
Quale dei seguenti UML state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?

Scegli un'alternativa:

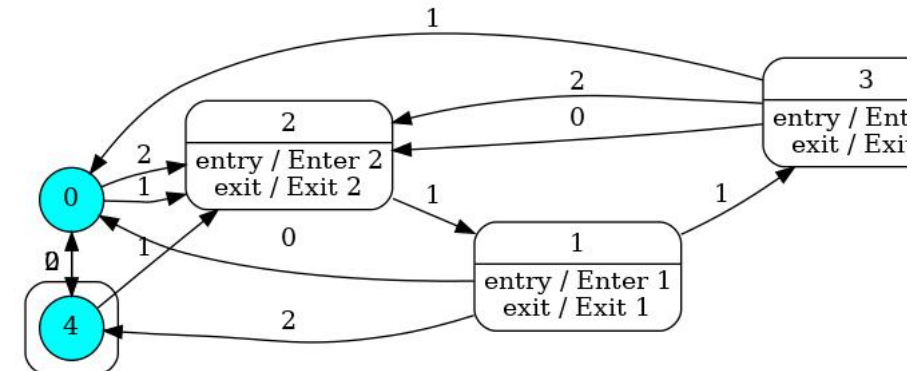
☒ 1.



☐ 2.

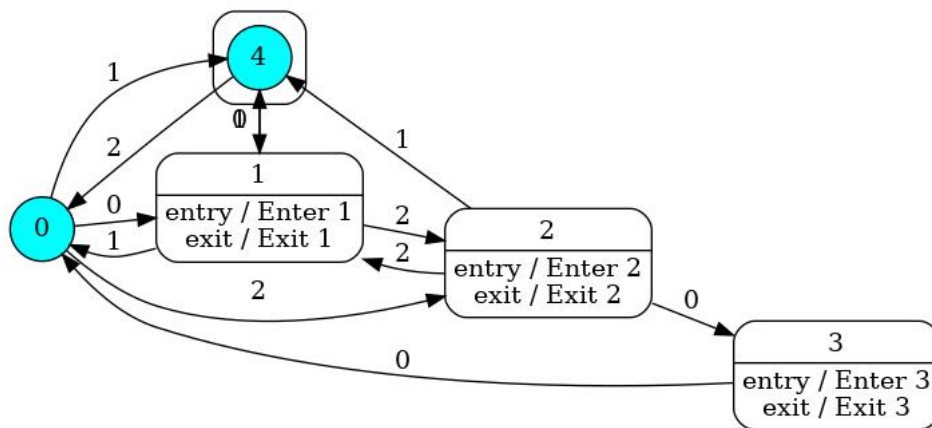


☐ 3.



Your answer is correct.

La risposta corretta è:



Domanda 20

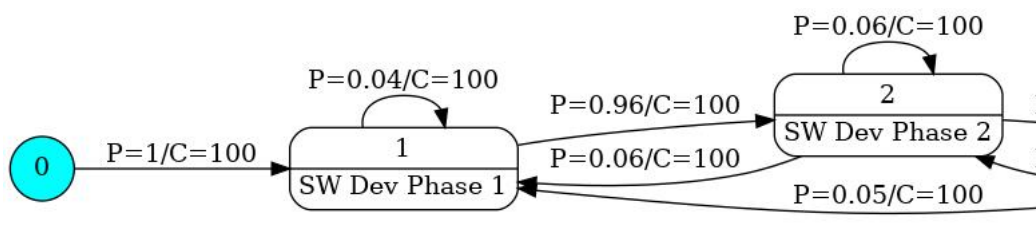
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.730147
- ☐ 2. 0.0425231
- ☒ 3. 0.0671 Correct.

Your answer is correct.

La risposta corretta è: 0.0671

Domanda 21

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri un processo software costituito da due fasi F1 ed F2 ciascuna di costo 10000. Con probabilità $p = 0.4$ la fase F1 deve essere ripetuta (a causa di change requests) e con probabilità $(1 - p)$ si passa alla fase F2 e poi al completamento (End) dello sviluppo. Qual'è il costo atteso per lo sviluppo del software seguendo il processo sopra descritto ?

Scegli un'alternativa:

- ☒ 1. 24000 ✓
☐ 2. 35000
☐ 3. 30000

Your answer is correct.

La risposta corretta è: 24000

Domanda 22

Risposta corretta

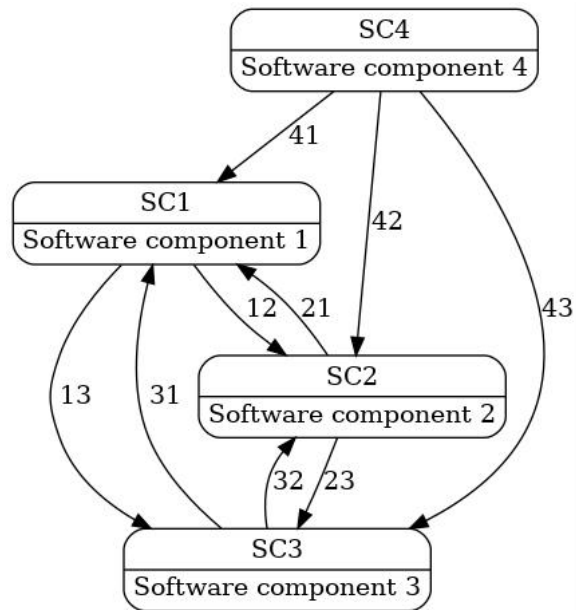
Punteggio ottenuto 1,00 su 1,00

Si consideri il seguente frammento di pseudo codice.

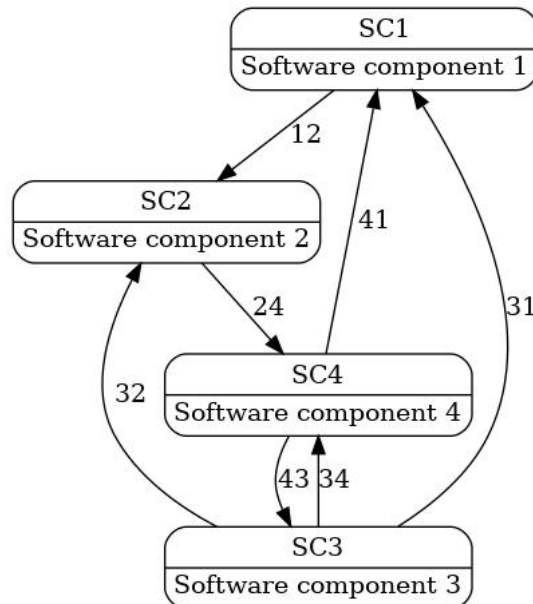
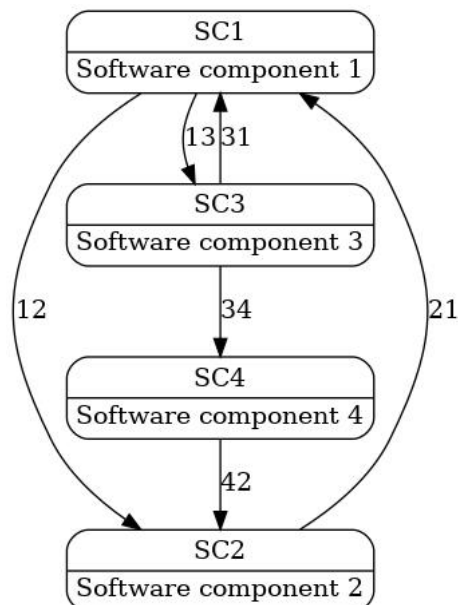
```
// Begin input/output connections for system components  
Input 12 of SC2 = Output 12 of SC1  
Input 13 of SC3 = Output 13 of SC1  
Input 21 of SC1 = Output 21 of SC2  
Input 23 of SC3 = Output 23 of SC2  
Input 31 of SC1 = Output 31 of SC3  
Input 32 of SC2 = Output 32 of SC3  
Input 41 of SC1 = Output 41 of SC4  
Input 42 of SC2 = Output 42 of SC4  
Input 43 of SC3 = Output 43 of SC4  
// End input/output connections for system components
```

Quale delle seguenti architetture software meglio lo rappresenta ?

Scegli un'alternativa:

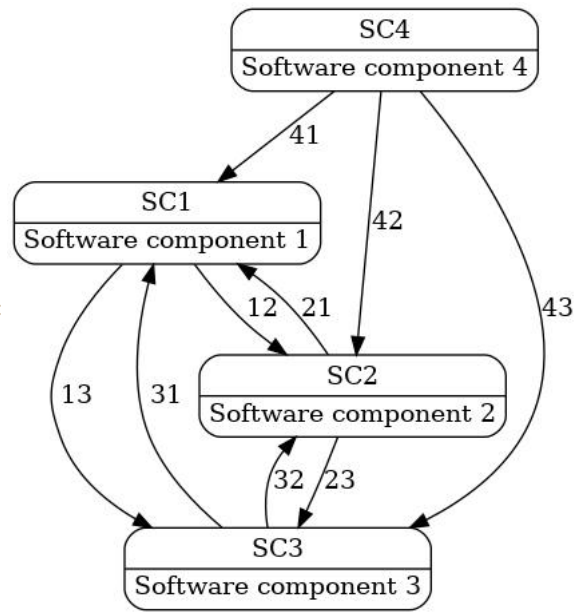
☒ 1.

✓ Correct.

☐ 2.☐ 3.

Your answer is correct.

La risposta corretta è:



Domanda 23

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "validity check" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.
- ☐ 2. Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.
- ☒ 3. Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del "customer". ✔

Your answer is correct.

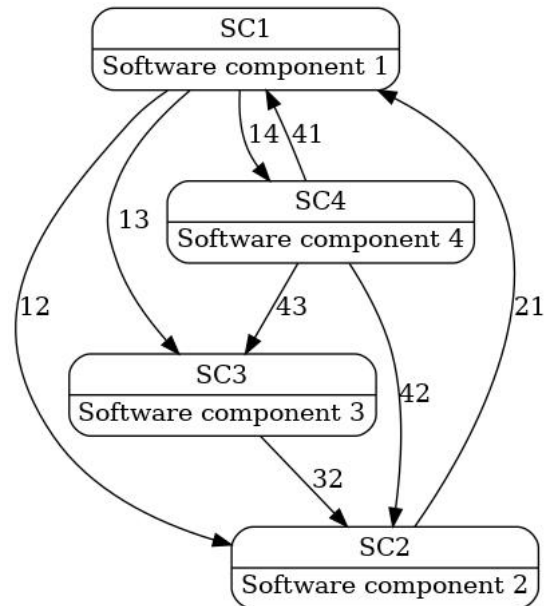
La risposta corretta è: Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del "customer".

Domanda 24

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente architettura software:



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Scegli un'alternativa:

- ☐ 1. // Begin input/output connections for system components
 Input 12 of SC2 = Output 12 of SC1
 Input 23 of SC3 = Output 23 of SC2
 Input 31 of SC1 = Output 31 of SC3
 Input 32 of SC2 = Output 32 of SC3
 // End input/output connections for system components
- ☐ 2. // Begin input/output connections for system components
 Input 13 of SC3 = Output 13 of SC1
 Input 14 of SC4 = Output 14 of SC1
 Input 21 of SC1 = Output 21 of SC2
 Input 23 of SC3 = Output 23 of SC2
 Input 31 of SC1 = Output 31 of SC3
 Input 32 of SC2 = Output 32 of SC3
 Input 42 of SC2 = Output 42 of SC4
 // End input/output connections for system components
- ☒ 3. // Begin input/output connections for system components ✔ Correct.
 Input 12 of SC2 = Output 12 of SC1
 Input 13 of SC3 = Output 13 of SC1
 Input 14 of SC4 = Output 14 of SC1
 Input 21 of SC1 = Output 21 of SC2
 Input 32 of SC2 = Output 32 of SC3
 Input 41 of SC1 = Output 41 of SC4
 Input 42 of SC2 = Output 42 of SC4
 Input 43 of SC3 = Output 43 of SC4
 // End input/output connections for system components

Your answer is correct.

La risposta corretta è: // Begin input/output connections for system components

Input 12 of SC2 = Output 12 of SC1

Input 13 of SC3 = Output 13 of SC1

Input 14 of SC4 = Output 14 of SC1

Input 21 of SC1 = Output 21 of SC2

Input 32 of SC2 = Output 32 of SC3

Input 41 of SC1 = Output 41 of SC4

Input 42 of SC2 = Output 42 of SC4

Input 43 of SC3 = Output 43 of SC4

// End input/output connections for system components

Domanda 25

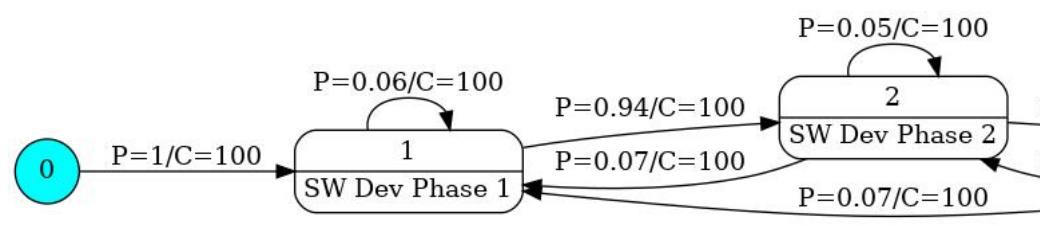
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.941135
- ☐ 2. 0.477053
- ☒ 3. 0.686576 ✔ Correct.

Your answer is correct.

La risposta corretta è: 0.686576

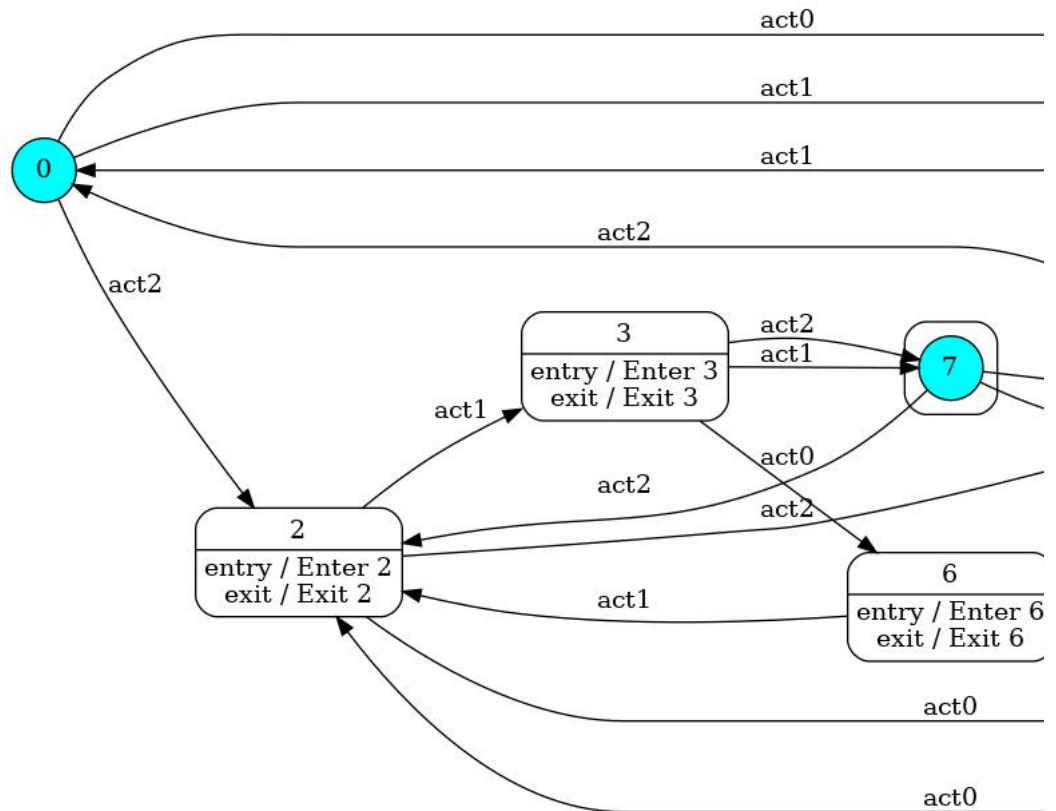
Domanda 26

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act0 act0 act2 act2 act0

Test case 2: act1 act0 act2 act2 act2

Test case 3: act1 act0 act2 act0 act0

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☒ 1. State coverage: 0.625 Correct.
- ☐ 2. State coverage: 0.361
- ☐ 3. State coverage: 0.210

Your answer is correct.

La risposta corretta è: State coverage: 0.625

Domanda 27

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il rischio R può essere calcolato come $R = P \cdot C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso.

Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = \exp(-b \cdot S)$, dove b è una opportuna costante note da dati storici aziendali.

- **$b = 0.000036$**
- **$C = 946574.000000$**
- **$R = 178.000000$**

Quale delle seguenti opzioni meglio approssima il costo S per lo sviluppo del software in questione.

Scegli un'alternativa:

- ☒ 1. Costo: 237969  Correct.
- ☐ 2. Costo: 155239
- ☐ 3. Costo: 255025

Your answer is correct.

La risposta corretta è: Costo: 237969

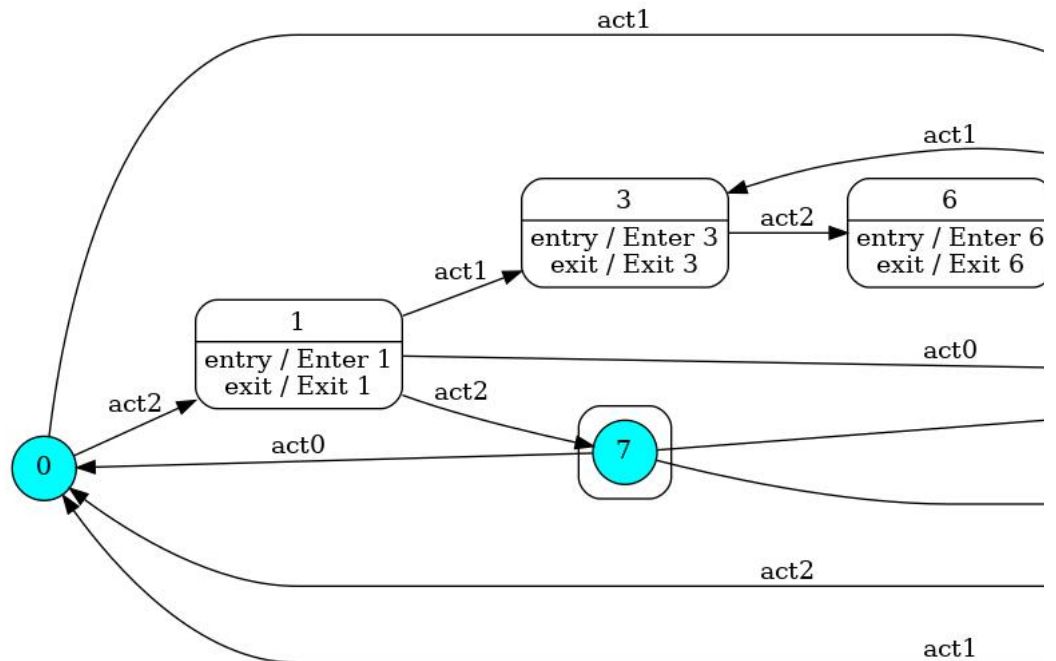
Domanda 28

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act1 act1 act2 act1 act0

Test case 2: act2 act0 act1 act0 act2

Test case 3: act2 act2

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☐ 1. State coverage: 0.397
- ☒ 2. State coverage: 1.000 Correct.
- ☐ 3. State coverage: 0.627

Your answer is correct.

La risposta corretta è: State coverage: 1.000

Domanda 29

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale dei seguenti è un requisito non funzionale?

Scegli un'alternativa:

- ☐ 1. L'output del sistema è sempre una matrice simmetrica.
- ☐ 2. L'output del sistema è sempre una potenza di 2.
- ☒ 3. Il tempo di esecuzione del sistema è inferiore al minuto.  Correct.

Your answer is correct.

La risposta corretta è: Il tempo di esecuzione del sistema è inferiore al minuto.

Domanda 30

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il team di sviluppo di un azienda consiste di un senior software engineer e due sviluppatori junior. Usando un approccio agile, ogni iterazione impegna tutti e tre i membri del team per un mese ed occorrono tre iterazioni per completare lo sviluppo. Si assuma che non ci siano "change requests" e che il membro senior costi A Eur/mese ed i membri junior B Eur/mese. Qual'è il costo dello sviluppo usando un approccio agile ?

Scegli un'alternativa:

- ☐ 1. $3*A + 2*B$
- ☒ 2. $3*(A + 2*B)$ 
- ☐ 3. $A + 2*B$

Your answer is correct.

La risposta corretta è: $3*(A + 2*B)$ **Domanda 31**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La WBS (Work Breakdown Structure) di un progetto consiste di 6 tasks: T0, T1, T2, T3, T4, T5.

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
1	0
2	0
3	1
3	2
4	3
5	3

La riga (*i*, *j*) nella tabella delle precedenze indica che il task *j* precede il task *i* (cioè che il task *i* può iniziare solo dopo che il task *j* è

terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
0	4
1	2
2	1
3	4
4	2
5	2

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling*. Ciascuna riga della tabella di scheduling consiste di una tripla (i, j, k) dove:

1. i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
2. j è il task
3. k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponedo di 4 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 4), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

☐ 1.

Time slot (Month)	Task	Effort (PM)
0	0	3
2	1	2
2	2	1
4	3	4
6	4	2
6	5	2

☐ 2.

Time slot (Month)	Task	Effort (PM)
0	0	4
2	1	2
2	4	2
3	2	1
4	3	4
6	5	2

☒ 3.

Time slot (Month)	Task	Effort (PM)
0	0	4
3	1	2
3	2	1
5	3	4
6	5	2
7	4	2

✔ Correct.

Your answer is correct.

La risposta corretta è:

Time slot (Month)	Task	Effort (PM)
0	0	4
3	1	2
3	2	1
5	3	4
6	5	2
7	4	2

Domanda 32

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

L'attività di *unit testing* di una funzione è stata suddivisa nei seguenti task:

Task identifier	Task description
TDEF	Definition of test cases
TORC	Definition of test oracle
TEXE	Esecuzione dei casi di test
TVAL	Valutazione dei casi di test

Quale dei seguenti insiemi di vincoli di precedenza (dove $A < B$ significa che A è preceduto da B) deve necessariamente valere, considerata la descrizione data dei tasks?

Scegli un'alternativa:

- ☐ 1. TVAL < TEXE, TORC < TDEF, TVAL < TORC.
- ☒ 2. TVAL < TEXE, TEXE < TDEF, TVAL < TORC. ☑ Correct.
- ☐ 3. TEXE < TORC, TVAL < TEXE, TVAL < TORC.

Your answer is correct.

La risposta corretta è: TVAL < TEXE, TEXE < TDEF, TVAL < TORC.

Domanda 33

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile $x1$ rappresenta lo stato precedente ad x e la variabile $x2$ lo stato precedente ad $x1$.

Si consideri il seguente requisito:

Se il sistema è nello stato 0 allora dopo al più due transizioni sarà nello stato 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( (x==0) ? (x2==2) : 0 )`
- ☒ b. `assert( (x2==0) ? (x==2) : 1 )` ☑
- ☐ c. `assert( (x==0) ? (x2==2) : 1 )`

Your answer is correct.

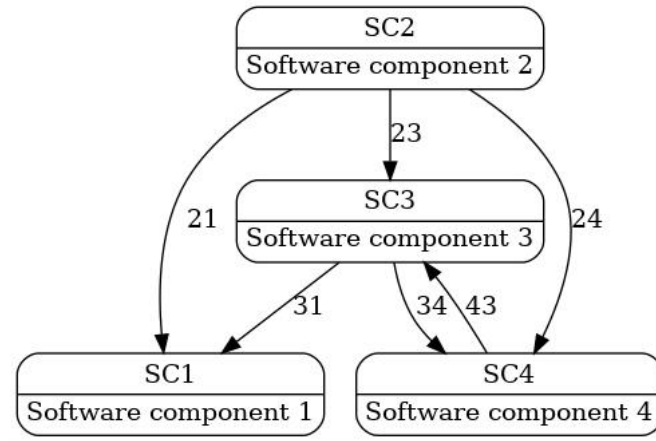
La risposta corretta è:
`assert( (x2==0) ? (x==2) : 1 )`

Domanda 34

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente architettura software:



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Scegli un'alternativa:

- ☐ 1. // Begin input/output connections for system components
 Input 12 of SC2 = Output 12 of SC1
 Input 21 of SC1 = Output 21 of SC2
 Input 23 of SC3 = Output 23 of SC2
 Input 31 of SC1 = Output 31 of SC3
 Input 32 of SC2 = Output 32 of SC3
 Input 34 of SC4 = Output 34 of SC3
 Input 41 of SC1 = Output 41 of SC4
 // End input/output connections for system components
- ☐ 2. // Begin input/output connections for system components
 Input 12 of SC2 = Output 12 of SC1
 Input 13 of SC3 = Output 13 of SC1
 Input 14 of SC4 = Output 14 of SC1
 Input 21 of SC1 = Output 21 of SC2
 Input 23 of SC3 = Output 23 of SC2
 Input 34 of SC4 = Output 34 of SC3
 Input 43 of SC3 = Output 43 of SC4
 // End input/output connections for system components
- ☒ 3. // Begin input/output connections for system components ✔ Correct.
 Input 21 of SC1 = Output 21 of SC2
 Input 23 of SC3 = Output 23 of SC2
 Input 24 of SC4 = Output 24 of SC2
 Input 31 of SC1 = Output 31 of SC3
 Input 34 of SC4 = Output 34 of SC3
 Input 43 of SC3 = Output 43 of SC4
 // End input/output connections for system components

Your answer is correct.

La risposta corretta è: // Begin input/output connections for system components

Input 21 of SC1 = Output 21 of SC2
 Input 23 of SC3 = Output 23 of SC2
 Input 24 of SC4 = Output 24 of SC2
 Input 31 of SC1 = Output 31 of SC3

Input 34 of SC4 = Output 34 of SC3
 Input 43 of SC3 = Output 43 of SC4
 // End input/output connections for system components

Domanda 35

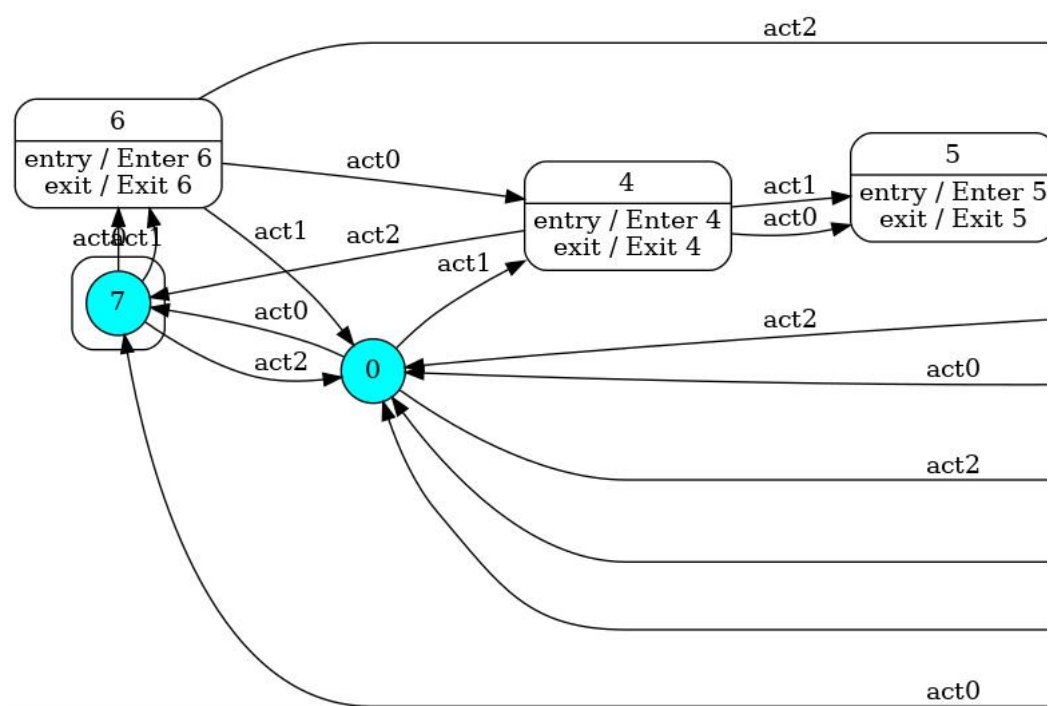
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram in figura:

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act1 act0 act2 act0

Test case 2: act2 act0 act2 act2

Test case 3: act1 act0 act1 act2 act1

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☒ 1. Transition coverage: 0.333 ✔ Correct.
- ☐ 2. Transition coverage: 0.874
- ☐ 3. Transition coverage: 0.220

Your answer is correct.

La risposta corretta è: Transition coverage: 0.333

Domanda 36

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x1 rappresenta lo stato precedente ad x e la variabile x2 lo stato precedente ad x1.

Si consideri il seguente requisito:

Se il sistema è nello stato 1 allora o permane nello stato di 1 oppure passa allo stato 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert((x1 == 1) ? ((x == 1) || (x == 2)) : 0)`
- ☐ b. `assert((x == 1) ? ((x1 == 1) || (x1 == 2)) : 1)`
- ☒ c. `assert((x1 == 1) ? ((x == 1) || (x == 2)) : 1)` ✓

Your answer is correct.

La risposta corretta è:

`assert((x1 == 1) ? ((x == 1) || (x == 2)) : 1)`

Domanda 37

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di completezza" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☒ 1. Assicurarsi che i requisiti descrivano tutte le funzionalità e vincoli (e.g., security, performance) del sistema desiderato dal customer. ✓
- ☐ 2. Assicurarsi che per ogni requisito sia stato implementato nel sistema.
- ☐ 3. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.

Your answer is correct.

La risposta corretta è: Assicurarsi che i requisiti descrivano tutte le funzionalità e vincoli (e.g., security, performance) del sistema desiderato dal customer.

Domanda 38

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

"Ogni giorno, per ciascuna clinica, il sistema genererà una lista dei pazienti che hanno un appuntamento quel giorno."

La frase precedente è un esempio di:

Scegli un'alternativa:

- ☐ 1. Requisito non-funzionale.
- ☒ 2. Requisito funzionale. ✓
- ☐ 3. Requisito di performance.

Your answer is correct.

La risposta corretta è: Requisito funzionale.

Domanda 39

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x_1 rappresenta lo stato precedente ad x e la variabile x_2 lo stato precedente ad x_1 .

Si consideri il seguente requisito:

Il valore assoluto della differenza tra il valore dello stato presente e di quello precedente è al più 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( (x - x1) <= 2)`
- ☐ b. `assert( abs(x - x1) >= 2)`
- ☒ c. `assert( abs(x - x1) <= 2)` ✓

Your answer is correct.

La risposta corretta è:
`assert( abs(x - x1) <= 2)`

Domanda 40

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Per testare un sistema si vuole costruire un generatore che ogni T secondi invia un valore v a sistema da testare.

Per ogni invio, il valore T è un valore intero scelto uniformemente a random nell'intervallo $[20, 30]$ mentre il valore v è un valore scelto aggiungendo, uniformemente a random, il valore -1 o $+1$ al valore precedente di v . Il valore iniziale di v è 0 .

Quale dei seguenti programmi meglio definisce il generator di cui sopra?

- ☒ a. `main{`
`srand(time(NULL));`
`v = 0;`
`while (1) {`
`send_value_to_system_under_testing(v);`
`T = 20 + rand()%11;`
`sleep(T);`
`v = v -1 + 2*rand()%2;`
`} // while`
`} // main()` 
- ☐ b. `main{`
`srand(time(NULL));`
`v = 0;`
`while (1) {`
`send_value_to_system_under_testing(v);`
`T = 20 + rand()%10;`
`sleep(T);`
`v = v + 1 - 2*rand()%2;`
`} // while`
`} // main()`
- ☐ c. `main{`
`srand(time(NULL));`
`v = 0;`
`while (1) {`
`send_value_to_system_under_testing(v);`
`T = 20 + rand()%11;`
`sleep(T);`
`v = -1 + 2*rand()%2;`
`} // while`
`} // main()`

Your answer is correct.

La risposta corretta è:

```
main{
srand(time(NULL));
v = 0;

while (1) {
send_value_to_system_under_testing(v);

T = 20 + rand()%11;
sleep(T);
v = v -1 + 2*rand()%2;

} // while

} // main()
```

Domanda 41

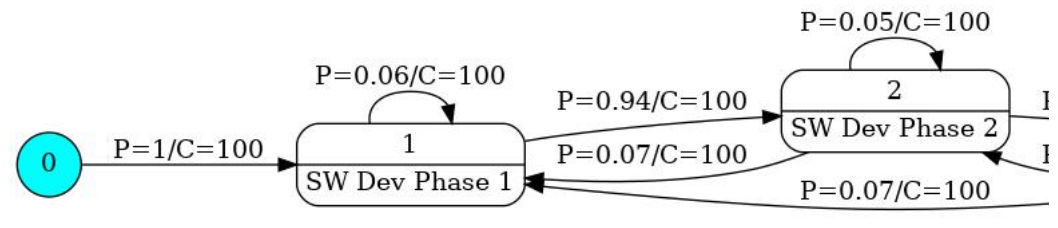
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.89041
- ☐ 2. 0.465602
- ☒ 3. 0.1095 Correct.

Your answer is correct.

La risposta corretta è: 0.1095

Domanda 42

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri un software sviluppato seguendo un approccio iterativo implementato con due fasi: F1 seguita da F2. Ciascuna fase ha costo 1000. Con probabilità 0.5 potrebbe essere necessario ripetere F1 una seconda volta. Con probabilità 0.2 potrebbe essere necessario ripetere F2 una seconda volta. Qual'è il costo atteso dello sviluppo dell'intero software?

Scegli un'alternativa:

- ☐ 1. 5000
- ☒ 2. 2700
- ☐ 3. 4000

Your answer is correct.

La risposta corretta è: 2700

Domanda 43

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti affermazioni è vera riguardo ai metodi agile ?

Scegli un'alternativa:

- ☒ 1. I metodi agile sono metodi di sviluppo incrementale. 
- ☐ 2. I metodi agile sono metodi di sviluppo plan-driven.
- ☐ 3. I metodi agile sono metodi di sviluppo orientato al riuso.

Your answer is correct.

La risposta corretta è: I metodi agile sono metodi di sviluppo incrementale.

Domanda 44

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Per testare un sistema si vuole costruire un generatore che ogni T secondi invia un valore v a sistema da testare.

Per ogni invio, il valore T è un valore intero scelto uniformemente a random nell'intervallo [1, 10] mentre il valore v è un valore intero scelto uniformemente a random nell'intervallo [0, 1].

Quale dei seguenti programmi meglio definisce il generator di cui sopra?

- ☒ a.

```
main{
    srand(time(NULL));
    while (1) {
        v = rand()%2;
        send_value_to_system_under_testing(v);
        T = 1 + rand()%10;
        sleep(T);
    } // while
} // main()
```


- ☐ b.

```
main{
    srand(time(NULL));
    while (1) {
        v = rand()%2;
        send_value_to_system_under_testing(v);
        T = rand()%10;
        sleep(T);
    } // while
} // main()
```

☐ c.

```
main{
    srand(time(NULL));
    T = 1 + rand()%10;
    v = rand()%2;
    while (1) {
        send_value_to_system_under_testing(v);
        sleep(T);
    } // while
} // main()
```

Your answer is correct.

La risposta corretta è:

```
main{
srand(time(NULL));
while (1) {
v = rand()%2;
send_value_to_system_under_testing(v);

T = 1 + rand()%10;
sleep(T);

} // while

} // main()
```

Domanda 45

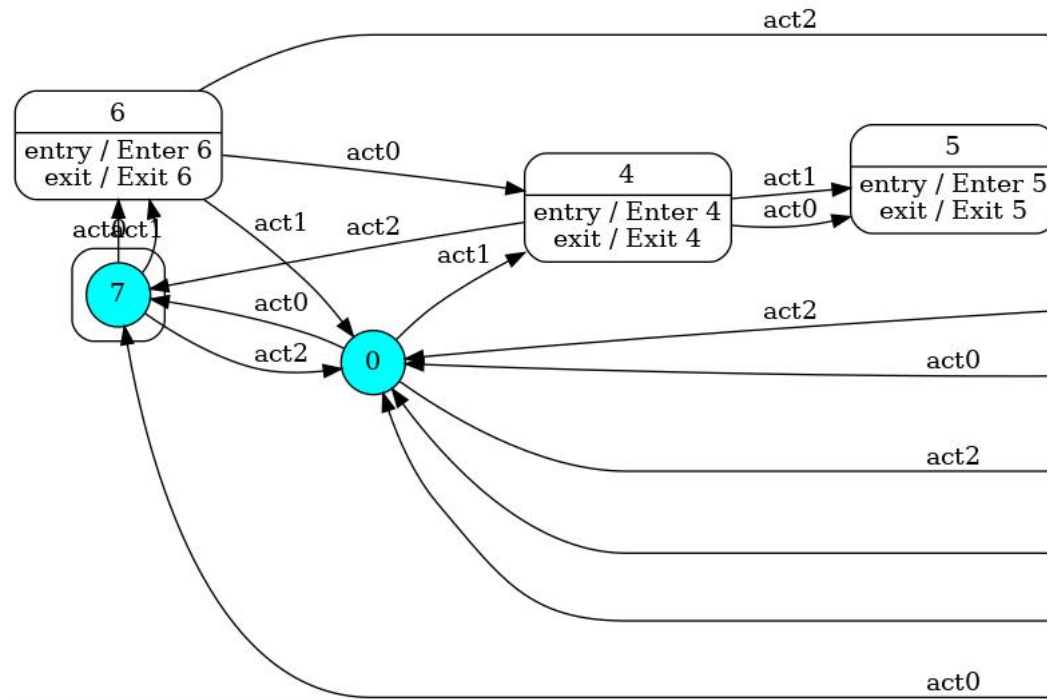
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram in figura:

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act1 act0 act2 act0

Test case 2: act2 act0 act2 act2

Test case 3: act1 act0 act1 act2 act1

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☐ 1. Transition coverage: 0.220
- ☒ 2. Transition coverage: 0.333 ✔ Correct.
- ☐ 3. Transition coverage: 0.874

Your answer is correct.

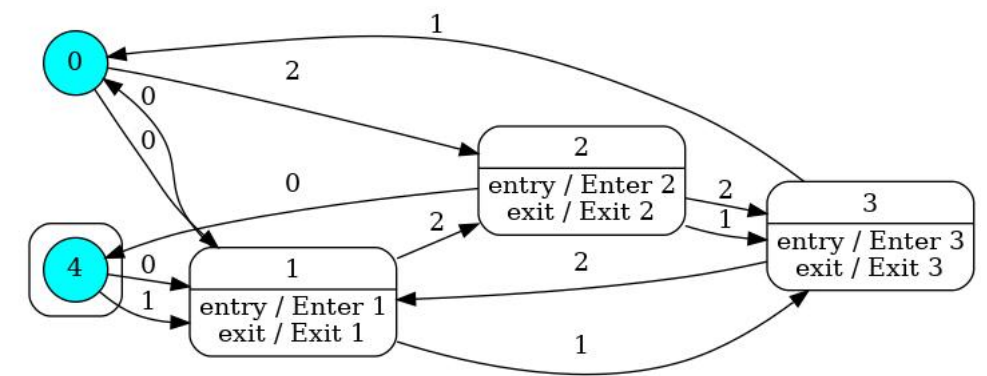
La risposta corretta è: Transition coverage: 0.333

Domanda 46

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri lo state diagram in figura.



Quale delle seguenti funzioni di transizione meglio lo rappresenta?

Scegli un'alternativa:

- ☒ 1. `#include "main.h"` ✔ Correct.

```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (1);
        case 1: return (0);
        case 2: return (2);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (0);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (3);
        case 2: return (3);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (3);
        case 1: return (0);
        case 2: return (1);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (4);
        default: return (0); }
    default: return (0);
}
} // f()
```

- ☐ 2. `#include "main.h"`

```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (3);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (0);
        case 1: return (4);
        case 2: return (1);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (3);
        case 1: return (3);
        case 2: return (4);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (2);
        case 2: return (0);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (3);
        case 1: return (3);
        case 2: return (1);
        default: return (0); }
    default: return (0);
}
} // f()
```

☐ 3. `#include "main.h"`

```
int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (2);
    case 1: return (0);
    case 2: return (4);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (2);
    case 1: return (0);
    case 2: return (1);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (3);
    case 1: return (1);
    case 2: return (0);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (3);
    case 1: return (0);
    case 2: return (0);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (2);
    case 1: return (1);
    case 2: return (2);
    default: return (0); }
  default: return (0);
}
} // f()
```

Your answer is correct.

`#include "main.h"`

```
int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (1);
    case 1: return (0);
    case 2: return (2);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (0);
    case 1: return (3);
    case 2: return (2);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (4);
    case 1: return (3);
    case 2: return (3);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (3);
    case 1: return (0);
    case 2: return (1);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (1);
    case 1: return (1);
    case 2: return (4);
    default: return (0); }
  default: return (0);
}
} // f()
```

La risposta corretta è:

Domanda 47

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Una azienda vende software utilizzando un contratto di Service Level Agreement (SLA) per cui l'utente paga 1000 Eur al mese di licenza e l'azienda garantisce che il software sia "up and running". Questo vuol dire che failures del software generano un costo (quello del repair). Sia $C = 10000$ Eur il costo del repair di una failure e $R = P \cdot C$ il valore atteso (rischio) del costo dovuto alle failures (dove P è la probabilità di una software failure). Ovviamente affinché il business sia profittevole deve essere che R sia al più 1000 Eur. Qual'e' il valore massimo di P che garantisce la validità del modello di business di cui sopra ?

Scegli un'alternativa:

- ☐ 1. $P = 1/1000$
- ☐ 2. $P = 1/10000$
- ☒ 3. $P = 1/10$ ✓

Your answer is correct.

La risposta corretta è: $P = 1/10$

Domanda 48

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.

La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la path coverage con i tests cases dati.

```
int f(int x[3])
{
    if ((-3)*x[0] + (-1)*x[1] + (3)*x[2] <= -2)
        if ((-3)*x[0] + (4)*x[1] + (3)*x[2] <= 1)
            {return (0); }
        else
            {return (1); }
    else if ((1)*x[0] + (-3)*x[1] + (3)*x[2] <= 1)
        if ((-1)*x[0] + (0)*x[1] + (-2)*x[2] <= -3)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */
```

Test 1: x[0] = 0, x[1] = 3, x[2] = -3.

Test 2: x[0] = 1, x[1] = -2, x[2] = -4.

Test 3: x[0] = -4, x[1] = 5, x[2] = -1.

Test 4: x[0] = 4, x[1] = -2, x[2] = 5.

Scegli un'alternativa:

- ☒ 1. Coverage: 0.800  Correct.
- ☐ 2. Coverage: 0.926
- ☐ 3. Coverage: 0.891

Your answer is correct.

La risposta corretta è: Coverage: 0.800

Domanda 49

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive il criterio di "requirements verifiability" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Per ciascuna coppia di componenti, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che l'interazione tra le componenti soddisfa tutti i requisiti di interfaccia.
- ☒ 2. Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che il sistema sviluppato soddisfa il requisito considerato. ✓
- ☐ 3. Per ciascuna componente del sistema, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che essa soddisfa tutti i requisiti.

Your answer is correct.

La risposta corretta è: Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che il sistema sviluppato soddisfa il requisito considerato.

Domanda 50

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La WBS (Work Breakdown Structure) di un progetto consiste di 6 tasks: T0, T1, T2, T3, T4, T5.

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
1	0
2	0
3	0
4	0
5	1
5	2
5	3
5	4

La riga (*i*, *j*) nella tabella delle precedenze indica che il task *j* precede il task *i* (cioè che il task *i* può iniziare solo dopo che il task *j* è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
0	5
1	1
2	5
3	6
4	6

5	6
---	---

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling*. Ciascuna riga della tabella di scheduling consiste di una tripla (i, j, k) dove:

1. i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
2. j è il task
3. k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponedo di 4 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 4), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

- ☒ 1.  Correct.

Time slot (Month)	Task	Effort (PM)
0	0	1
1	0	4
2	1	1
2	4	3
3	2	1
3	4	3
4	2	4
5	3	4
6	3	2
7	5	4
8	5	2

○ 2.

Time slot (Month)	Task	Effort (PM)
0	1	1
2	0	1
3	0	4
4	2	3
4	4	1
5	2	2
5	3	2
6	4	1
7	4	4
8	3	4
9	5	4
10	5	2

○ 3.

Time slot (Month)	Task	Effort (PM)
0	0	1
1	0	4
2	3	4
3	1	1
3	4	3
4	2	3
4	4	1
5	3	2
5	4	2
6	2	1
7	5	2
8	5	4

Your answer is correct.

La risposta corretta è:

Time slot (Month)	Task	Effort (PM)
0	0	1
1	0	4
2	1	1
2	4	3
3	2	1
3	4	3
4	2	4
5	3	4
6	3	2
7	5	4
8	5	2